OpenWork Python AI Service

FastAPI micro-service that powers AI features for the OpenWork freelance marketplace. Runs **entirely on free OpenRouter models** — no paid keys required.

Features

Endpoint	Method	What it does
`/ai/health`	GET	Liveness + shows which primary model is configured
`/ai/chat`	POST	Career-assistant chat for freelancers
`/ai/generate-proposal`	POST	Drafts a concise proposal from a job description and freelancer profil
`/ai/job-match`	POST	Scores freelancer job fit with TF-IDF cosine similarity + experience
`/ai/skill-test/generate`	POST	Generates MCQs for a topic at a given level (AI + built-in fallback b
`/ai/skill-test/evaluate`	POST	Grades free-text answers with the LLM; returns per-question scores
`/ai/fraud-detect`	POST	Anomaly score over `[loginPatterns, bidAmounts, responseTimes`
`/ai/skill-suggestions`	POST	Returns in-demand skills for a category (AI-curated with a static fall
`/ai/moderate`	POST	**Returns `SAFE` or `UNSAFE`** for a single message — flags contact-detai

OpenAPI docs are auto-generated at `/docs` and `/redoc`.

Architecture

```
openwork-ai-service/
  app/
      main.py                    # FastAPI app assembly + CORS
      config.py                  # Env-driven settings (models, keys, ports)
      schemas.py                 # Pydantic request/response models
      data/
          fallback.py           # Built-in quiz bank used when AI is unavailable
      services/
          openrouter.py         # LLM client with multi-model fallback
          json_utils.py         # Robust JSON extraction from LLM output
          moderation.py         # Regex + LLM SAFE/UNSAFE classifier
      routers/                   # One file per endpoint family
  scripts/
      run_dev.sh                 # Creates venv, installs deps, starts reload server
      smoke_test.sh              # curls every endpoint
  tests/
      test_offline.py            # pytest suite that runs without an API key
  Dockerfile
  Procfile                       # for Railway / Heroku-style platforms
  render.yaml                    # one-click Render.com deploy
  requirements.txt
```

```
.env.example
README.md
```

Every LLM call goes through `app/services/openrouter.py`, which:

1. Uses the **OpenAI SDK** pointed at `https://openrouter.ai/api/v1`.
2. Tries a **chain of free models** in order (configurable via `OPENROUTER_MODELS`). The defaults are:
   - `openai/gpt-oss-120b:free`
   - `nvidia/nemotron-3-super-120b-a12b:free`
   - `meta-llama/llama-3.3-70b-instruct:free`
   - `qwen/qwen3-next-80b-a3b-instruct:free`
   - `openrouter/free` (OpenRouter's own auto-router across free models)
3. Returns the caller-provided **fallback string** only if every model fails or `OPENROUTER_API_KEY` is missing.

That design is why the service **degrades gracefully** instead of returning empty responses the way the original code did.

Setup (local)

Prereqs: Python 3.11+.

```
git clone <this repo> openwork-ai-service
cd openwork-ai-service

# option A - use the convenience script (creates venv, installs deps, starts server)
chmod +x scripts/run_dev.sh scripts/smoke_test.sh
./scripts/run_dev.sh

# option B - manual
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
# Edit .env and set OPENROUTER_API_KEY
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Grab a free key from https://openrouter.ai/keys and paste it into `.env`.

Smoke-test every endpoint

With the server running on `http://localhost:8000`:

```
./scripts/smoke_test.sh
```

Run the test suite

These tests **do not need an API key** — the LLM layer is stubbed via the fallback path.

```
pip install pytest
pytest -q
```

Environment variables

Variable	Default
OPENROUTER_API_KEY	*(required for live AI)*
OPENROUTER_MODELS	openai/gpt-oss-120b:free,nvidia/nemotron-3-super-120b-a12b:free,meta-l
OPENROUTER_BASE_URL	https://openrouter.ai/api/v1
OPENROUTER_APP_URL	https://openwork.local
OPENROUTER_APP_NAME	OpenWork AI Service
HOST	0.0.0.0
PORT	8000
LOG_LEVEL	INFO

Example requests

```
# Moderation: SAFE
curl -sS -X POST http://localhost:8000/ai/moderate \
  -H "Content-Type: application/json" \
  -d '{"message":"Hi! Excited to work together via OpenWork."}'
# -> {"verdict":"SAFE","reason":"no contact indicators detected"}

# Moderation: UNSAFE (email shared)
curl -sS -X POST http://localhost:8000/ai/moderate \
  -H "Content-Type: application/json" \
  -d '{"message":"Email me at jane@acme.com and we can skip the platform."}'
# -> {"verdict":"UNSAFE","reason":"email address detected"}
```

See `scripts/smoke_test.sh` for one example payload per endpoint.

Deployment

Docker

```
docker build -t openwork-ai-service .
docker run -p 8000:8000 --env-file .env openwork-ai-service
```

Render.com (free tier)

1. Push the repo to GitHub.

2. On Render → *New +* → *Blueprint* → select the repo. Render picks up `render.yaml`.
3. In *Environment*, set `OPENROUTER_API_KEY`.
4. Deploy. Your service URL will be `https://<name>.onrender.com`; health at `/ai/health`.

Railway / Fly.io / Heroku-style platforms

`Procfile` is already provided (`uvicorn app.main:app --host 0.0.0.0 --port $PORT`). Set `OPENROUTER_API_KEY` in the platform's secrets UI.

Vercel / Netlify

These platforms are tuned for front-end and serverless functions; a long-lived FastAPI process is better suited to Render, Railway, Fly.io, or a container host. If you must use Vercel, deploy the frontend there and point it at this API hosted elsewhere.

Packaging as a downloadable ZIP

From the repo root:

```
zip -r openwork-ai-service.zip . \
  -x ".venv/*" "**/__pycache__/*" ".git/*" ".env" "*.pyc"
```

A pre-built zip is also provided alongside this README when delivered by the generator.

Security notes

- `.env` is gitignored. Never commit real keys.
- The moderation endpoint treats mentions of ”OpenWork” as safe context; handle allow-listing is in `app/services/moderation.py`.
- CORS is currently `*` to keep dev frictionless — lock it down in `app/main.py` before production.